THE INTERVIEW

HIGH-LEVEL DESIGN DOCUMENT

1. Introduction

1.1 Project Overview

THE INTERVIEW is an AI-driven mock interview and technical assessment platform designed to bridge the gap between candidate preparation and real-world hiring standards. The platform leverages Generative AI (Google Gemini) and a microservices-based architecture to provide a realistic, scalable, and proctored interview environment.

1.2 Objectives

- **Scalability:** Handle high concurrent user traffic via a distributed microservices architecture.
- **Reliability:** Ensure system availability and eventual consistency using event-driven messaging (RabbitMQ).
- **Intelligence:** Provide dynamic, non-repetitive interview experiences using the Gemini 2.5-Flash model.
- **Security:** Protect user data and maintain integrity via JWT-based authorization and API Gateway routing.

2. Technology Stack

Layer	Technology	Details
Frontend	Angular v21.2.0	RxJS, HTML5, standalone components, component CSS
Backend	.NET Web API v10.0 (net10.0)	Entity Framework Core v10.0
Gateway	Ocelot API Gateway	Centralized Routing & JWT Enforcement
Database	MS SQL Server	Distributed Databases per service
Messaging	RabbitMQ	Asynchronous Event-Driven Bus
AI Engine	Google Gemini API	Model: gemini-2.5-flash
Payments	Stripe API	Secure Checkout & Webhook Sync

3. High-Level Architecture

3.1 Architecture Overview

THE INTERVIEW follows a microservices-based architecture. The system is decomposed into specialized services that communicate asynchronously through a message broker, ensuring high decoupling and fault tolerance.

Key Architectural Patterns

- API Gateway Pattern — Ocelot serves as the single entry point, handling cross-cutting concerns like authentication and routing.
- Database-per-Service — Each service manages its own data persistence layer to prevent coupling.
- Saga Pattern (Event-Driven) — Orchestrates distributed transactions across services (e.g., synchronizing premium status after a successful payment).

3.2 System Architecture Diagram

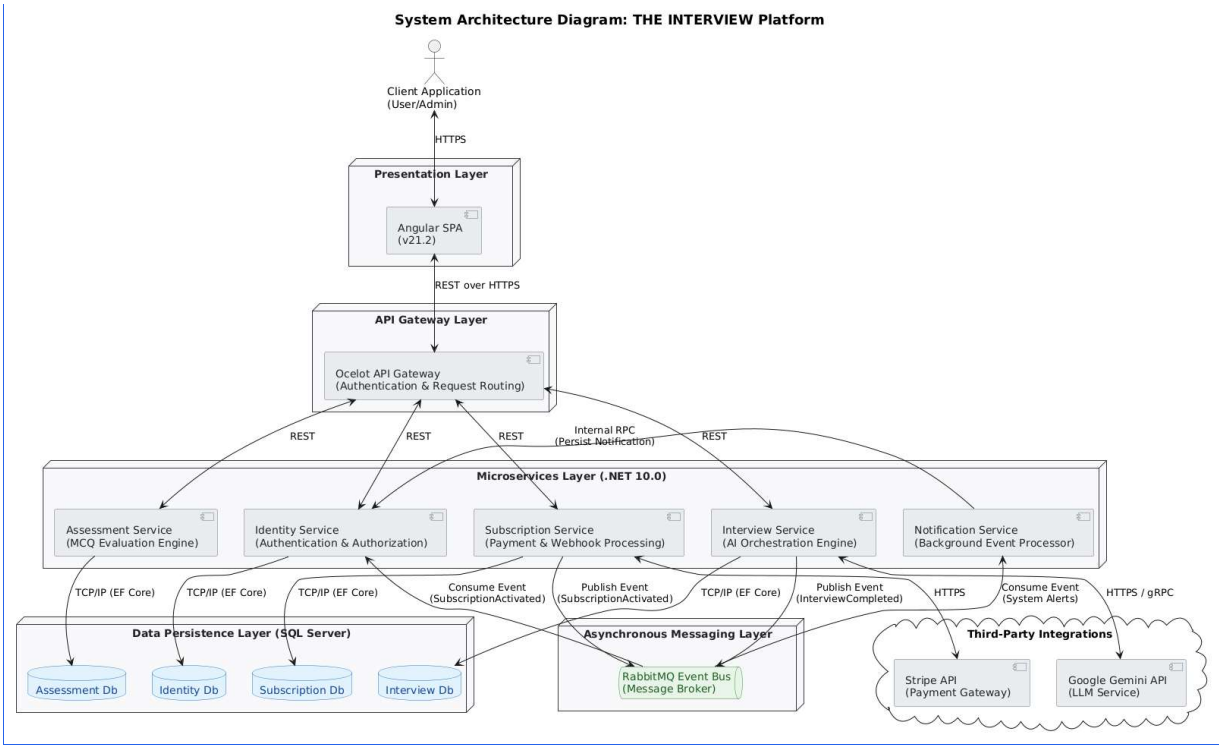


Figure 3.1 — System Architecture Diagram (Cloud-Native Microservices)

4. System Components

4.1 Client Layer

01 Candidate Dashboard

- Angular-based Single Page Application (SPA)
- Provides a seamless UI for interviews and assessments
- Built with RxJS, Angular standalone components, and component-level CSS for responsive design

02 Admin Portal

- Management interface for users, analytics, and question banks
- Role-protected routes via JWT claim validation
- Question bank management through admin question CRUD screens

4.2 Gateway Layer

03 Ocelot API Gateway
▸ Routes all incoming client requests to the correct upstream microservice
▸ Validates JWT claims and enforces role-based authorization
▸ Provides a single stable entry point — clients never call services directly
▸ Handles cross-cutting concerns: centralized routing, JWT validation, Swagger aggregation, and selected QoS timeout settings

4.3 Service Layer

04 Identity Service
▸ Manages user registration, login, and JWT issuance
▸ Handles Role-Based Access Control (RBAC) for User and Admin roles
▸ Maintains user identity state: full name, email, role, active status, and premium status
▸ Password stored using BCrypt hashing

05 Interview Service
▸ Orchestrates AI-generated mock interview sessions via Gemini API
▸ Supports Gemini-generated interview questions with cached/global question reuse where available
▸ Handles voice answers via Speech-to-Text integration
▸ Triggers evaluation and persists feedback and scores to its own database

06 Assessment Service
▸ Manages domain-specific MCQ question banks with admin CRUD and Gemini-generated question caching
▸ Supports dynamic AI-generated question sets via Gemini API
▸ Supports proctored sessions using fullscreen mode, tab-switch tracking, warnings, and auto-submission
▸ Auto-saves draft answers and auto-submits on proctoring violation limit

07 Subscription Service
▸ Manages Stripe checkout session creation and webhook processing
▸ Triggers the Premium Saga via RabbitMQ on successful payment confirmation
▸ Synchronizes premium status with IdentityService so refreshed JWT claims include isPremium
▸ Enforces free-tier limits and premium access through subscription status and JWT premium claims

08 Notification Service
▸ Background worker service for RabbitMQ-driven email notifications
▸ Subscribes to RabbitMQ events for asynchronous notification dispatch
▸ Sends email notifications through MailKit/SMTP; in-app notifications are stored by IdentityService consumers

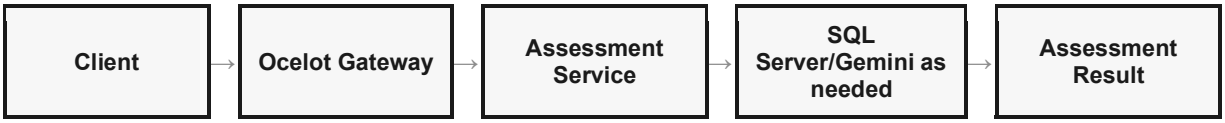
4.4 External Integration Layer

External Integrations	
Google Gemini API	Provides LLM capabilities for dynamic question generation, answer evaluation, and AI feedback. Model: gemini-2.5-flash.
Stripe API	Handles secure global payments, PCI-compliant checkout sessions, and webhook-based subscription sync.
Speech-to-Text	Converts voice interview responses to text for evaluation processing by the AI engine.

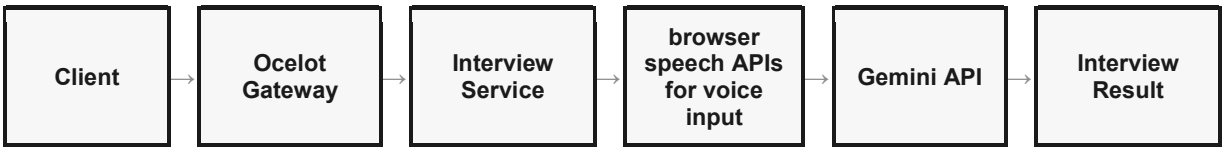
5. Data Flow Overview

5.1 Authentication Flow

5.2 Assessment Flow



5.3 Interview Flow



5.4 Payment Flow

6. Security Architecture

Security Controls	
Authentication	JWT (JSON Web Tokens) used for stateless, scalable authentication between client and gateway.
Token Signing	Symmetric encryption with shared secret keys for JWT signing and verification.
Internal API Security	Service-to-service communication protected via InternalApiKey header validation.
Password Storage	BCrypt hashing with salt rounds — plaintext passwords are never stored.
Payment Security	Stripe webhook signature verification ensures only legitimate events trigger subscription activation.
Proctoring Security	Frontend-driven anti-cheat mechanisms including tab-switch detection, fullscreen mode, warnings, and auto-submission on repeated violations.
Input Validation	DTO validation and controlled API inputs are used; file-upload validation can be added if upload features are introduced.

7. User Roles

7.1 Candidate (User)

- Attempt domain-specific MCQ assessments using cached and Gemini-generated questions
- Conduct Gemini-powered mock interviews with text and voice answer input
- Purchase premium subscription and manage premium access
- View detailed results, feedback, and performance analytics
- Manage profile basics such as full name and account details

7.2 Administrator

- Add, edit, and delete questions from the question bank
- Add, edit, and delete questions through admin question management screens
- View and manage registered users
- View admin dashboards, users, interviews, assessments, subscriptions, and payment records
- Manage platform content and configuration

8. Non-Functional Requirements

NFR Summary	
Performance	Fast API response times; AI calls use retry/fallback handling and lazy/background question loading where implemented to reduce user-facing delays.
Scalability	Microservices architecture enables horizontal scaling of individual services under high load.
Reliability	Session storage preserves active interview/assessment progress on the frontend; final persistence happens on submission.
Availability	JWT-based APIs and gateway routing support service isolation; production zero-downtime deployment would require deployment infrastructure not included in this repo.
Maintainability	Database-per-service isolation allows teams to update, test, and deploy services independently.
Usability	Simple, guided user flows reduce onboarding friction for first-time candidates.

9. Deployment Overview

9.1 Infrastructure

Deployment Stack	
Frontend	Angular application — compiled SPA served via Nginx or CDN.
Backend	ASP.NET Core microservices targeting net10.0, runnable as separate services.
Database	Microsoft SQL Server — one isolated instance per service.
Message Broker	RabbitMQ — managed instance for event-driven communication.
AI Engine	Google Gemini API — external managed service via HTTPS.
Payments	Stripe API — external PCI-compliant payment infrastructure.

9.2 Deployment Flow

10. Future Scope

The following capabilities are planned for future releases:

- Live coding interview environment with real-time code execution and review
- Video interview recording with AI-driven behavioral analysis
- Automated resume parsing and AI-powered career gap analysis
- Company-specific interview simulations based on known interview patterns
- AI-generated personalized career roadmap and skill gap recommendations
- Advanced analytics dashboard with cohort comparison and performance trends

11. Conclusion

THE INTERVIEW is designed as a scalable AI-powered interview preparation platform built on a microservices architecture. It combines structured assessments, AI-driven evaluations, browser-level proctoring mechanisms, and premium subscription features into a unified platform — all orchestrated through a secure microservices architecture built on .NET, Angular, Gemini AI, and Stripe.

The platform is engineered to handle real-world scale, evolve modularly, and deliver consistently intelligent and personalized interview preparation experiences to every candidate.